

Relatório de Auditoria de Segurança

Meeting Scheduler — Booking SaaS multi-tenant (event_service_report)

Data do relatório	31/08/2026
Repositório	meeting-scheduler / event_service_report
Branch auditada	main
Stack detectada	FastAPI + SQLAlchemy async (asynpg) + Postgres 16, JWT (sem papéis) — backend novo; React + TypeScript + Vite — frontend novo; Docker Compose + nginx + GitHub Actions — deploy; Node.js/Vercel + Supabase (legado, dashboard/ e booking-site/)
Escopo	Código-fonte completo em src/backend e src/frontend (stack novo), o app legado Supabase em dashboard/ e booking-site/, arquivos de infraestrutura (docker-compose.yml, infra/nginx, .github/workflows) e histórico completo do Git

Nota metodológica

Cada uma das cinco categorias solicitadas foi mapeada para o mecanismo equivalente na stack detectada, dado que o repositório contém dois sistemas distintos: (1) um backend/frontend novo em FastAPI + Postgres + React, sem RLS nativa — isolamento de tenant é feito por filtro explícito de tenant_id na camada de serviço/repositório; e (2) um app legado em substituição, que usa Supabase diretamente do navegador (onde o mecanismo de isolamento seria Row Level Security). 'Banco sem tranca' foi avaliado como RLS ausente/furada no app Supabase e como ausência/presença de filtro de tenant_id explícito no app FastAPI. 'Permissão definida no navegador' foi avaliado considerando que a stack nova não possui sistema de papéis (User não tem coluna role — todo usuário autenticado é administrador do seu próprio tenant), portanto não há gate de papel frontend↔backend para cruzar; documentado explicitamente como não aplicável nesse sentido. 'IDOR' foi verificado em todos os handlers de rota dos dois backends, um por um. 'Chaves expostas' incluiu uma varredura do código-fonte atual E do histórico completo do Git (git log --all), já que segredos removidos de commits recentes podem permanecer recuperáveis. 'Inputs sem tratamento (XSS)' cobriu o React novo (dangerouslySetInnerHTML/innerHTML/eval) e a geração de HTML server-side do app legado (booking-site/api/_lib/renderPage.js).

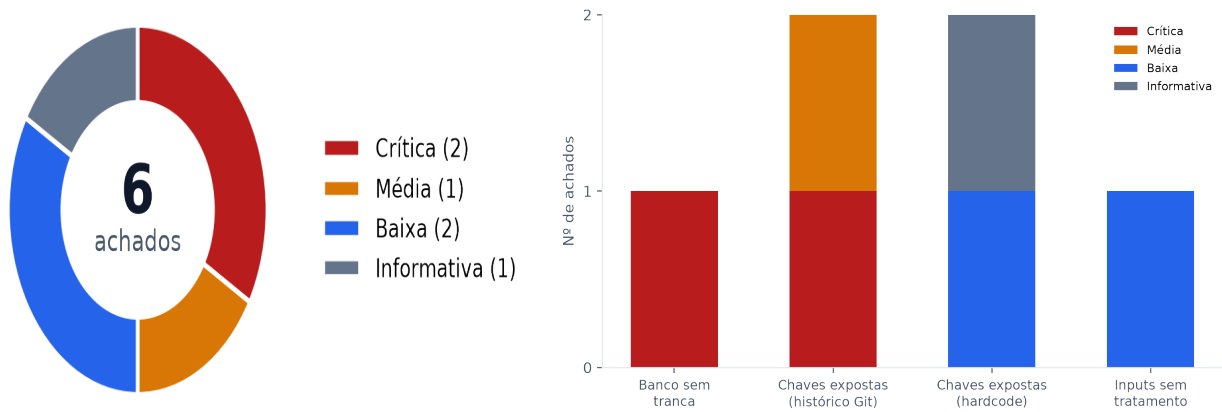
Uso restrito. Este relatório contém referências a segredos comprometidos (Seção b/d) — os valores reais foram mascarados; a evidência completa está disponível nos commits do Git referenciados. Recomenda-se tratar este PDF com a mesma confidencialidade do repositório.

Resumo executivo

2 Crítica	0 Alta	1 Média	2 Baixa	1 Informativa
---------------------------------	------------------------------	-------------------------------	-------------------------------	-------------------------------------

Total de 6 achados classificados, além de 11 pontos fortes verificados com evidência de código. O risco central da auditoria concentra-se inteiramente no app legado em substituição (dashboard/ e credenciais commitadas no histórico do Git) — a stack nova (FastAPI/Postgres/React) não apresentou achados de isolamento de tenant, IDOR ou XSS.

Distribuição por severidade e por categoria



Pontos fortes

Práticas de segurança verificadas no código, com a evidência que comprova cada uma — isso também demonstra a cobertura desta auditoria.

- **Isolamento por tenant_id aplicado de forma consistente em toda a stack nova** [Banco sem tranca / isolamento de tenant]

Toda tabela multi-tenant (users, customers, services, agendamentos, notifications, refresh_tokens via user_id) tem tenant_id e toda query em repository.py (agendamentos, customers, services, notifications, companies) filtra explicitamente por tenant_id — nunca por join implícito. tenant_id vem sempre de current_user.tenant_id (JWT), nunca do corpo/URL da requisição, em agendamentos.py, services.py, customers.py, companies.py.

- **Constraint de exclusão no Postgres cobre a janela de corrida do check de disponibilidade** [Banco sem tranca / concorrência]

ex_agendamentos_no_overlap (models.py:150-160, migração b7c4f19a2e30) impede duas reservas pending/confirmed sobrepostas por tenant mesmo sob concorrência, com fallback tratado como 409 em agendamentos/service.py:57-67.

- **Todos os 8 routers autenticados revisados linha a linha não apresentam IDOR** [IDOR]

agendamentos.py, services.py, customers.py, companies.py e notifications.py resolvem objeto-por-id sempre através de get_*(db, tenant_id, id) no domínio correspondente, que retorna 404 (não 403) quando o id pertence a outro tenant — testado explicitamente em test_services_router.py::test_update_other_tenants_service_404s e test_delete_other_tenants_service_404s.

- **Confirmação/recusa de agendamento no booking-site protegida por token HMAC com comparação de tempo constante** [IDOR (app legado)]

api/_lib/token.js usa HMAC-SHA256 assinado com BOOKING_TOKEN_SECRET e crypto.timingSafeEqual — um id de booking sozinho (mesmo sequencial/adivinhável) não é suficiente para confirmar/recusar outra reserva.

- **Não há sistema de papéis nem em frontend nem em backend — não há gate para cruzar**

[Permissão definida no navegador]

core/models.py::User não tem coluna role; ProtectedRoute.tsx só verifica presença de usuário autenticado; toda rota staff exige get_current_user no backend (main.py:30-33). Não há isAdmin/canEdit em nenhum componente do frontend novo.

- **Nenhuma credencial de produção hardcoded no código-fonte atual da stack nova** [Chaves expostas]

core/config.py exige SECRET_KEY, POSTGRES_PASSWORD via variáveis de ambiente sem valor padrão (Settings()) falha ao subir se ausente); .env real está no .gitignore e nunca foi commitado; apenas .env.example com placeholders 'changeme' está versionado.

- **Frontend novo não usa nenhuma API de renderização de HTML bruto** [Inputs sem tratamento (XSS)]

Busca por dangerouslySetInnerHTML/innerHTML/eval/new Function/v-html em todo src/frontend/src retornou zero ocorrências — toda renderização passa pelo escaping automático do React (JSX).

- **Página HTML gerada pelo app legado (booking-site) escapa toda entrada do usuário**

[Inputs sem tratamento (XSS)]

api/_lib/renderPage.js:1-8 define escapeHtml() e a aplica em todo valor interpolado (title, message, detalhes do cliente/serviço/telefone) antes de montar a página de confirmar/recusar.

● **Content-Security-Policy estrita e cabeçalhos de segurança completos no nginx** [Inputs sem tratamento (XSS) / hardening de rede]

infra/nginx/meeting-scheduler.conf:56-67 define CSP (default-src 'self', script-src 'self', object-src 'none', frame-ancestors 'none'), HSTS, X-Content-Type-Options, X-Frame-Options e Referrer-Policy em todas as localizações relevantes; rate limiting dedicado em /api/auth/login e /api/auth/refresh.

● **Refresh token com revogação real no servidor e cookie httpOnly** [Autenticação]

RefreshToken (models.py) com jti/revoked_at/expires_at verificados em auth/service.py::rotate_refresh_token; cookie httpOnly+SameSite=strict com path restrito a /api/auth (core/auth.py:55-64); access token mantido só em memória no frontend (auth.ts:7-9), nunca em localStorage.

● **Suíte de testes automatizados cobre isolamento cross-tenant** [Cobertura de testes]

14 arquivos de teste em src/backend/tests/, incluindo casos explícitos de acesso cross-tenant retornando 404 (test_services_router.py) e testes de fluxo completo de auth/refresh.

Pontos fracos (riscos centrais)

- Isolamento de tenant no app Supabase legado depende inteiramente de RLS, que está desabilitada — qualquer chamada direta à API do Supabase com a anon key ignora o filtro de tenant da UI (F1).
- Segredos reais (OAuth do Google) foram commitados e continuam recuperáveis do histórico do Git mesmo após remoção em commits posteriores — a equipe já havia identificado isso internamente, mas não há confirmação de rotação (F2).
- A chave pública do Supabase, combinada com a ausência de RLS, funciona como uma chave mestra de acesso a todas as tabelas do app legado (F3).

Achados detalhados

Sev.	Arquivo:linha	Descrição
Crítica	dashboard/src/App.jsx:192-194 dashboard/src/supabaseClient.js:1-6	F1 — Painel legado (dashboard/) lê dados de todos os tenants sem autenticação e sem RLS O painel administrativo legado (Supabase) não possui nenhuma tela de login nem chamada a supabase.auth — carrega os dados assim que a página abre. As três queries usam a anon key pública diretamente do navegador e filtram por tenant apenas no JavaScript do cliente (.eq('project_id', PROJECT_ID)), nunca no servidor:
Crítica	credentials_anabela.json (commit 164efb6) token_anabela.json (commit 164efb6)	F2 — Credenciais reais do Google OAuth (client_secret + refresh_token) commitadas e ainda recuperáveis no histórico do Git O commit 164efb6 ("first commit") adicionou os arquivos credentials_anabela.json e token_anabela.json com um client_secret OAuth real do Google Cloud (projeto 'cabeleireiro-calendario') e um refresh_token de escopo calendar.readonly. Os arquivos foram removidos de commits posteriores e hoje estão no .gitignore (credential_token/, credentials_*.json), mas o histórico do Git NÃO foi reescrito — 'git show 164efb6:credentials_anabela.json' ainda recupera o segredo integralmente para qualquer pessoa com acesso de leitura ao repositório.
Média	.env (commit 224ef52) dashboard/.env.local (commit f4a3370)	F3 — Chave pública (anon/publishable) e URL do Supabase commitadas no histórico do Git VITE_SUPABASE_URL e VITE_SUPABASE_ANON_KEY (sb_publishable...) foram commitados em .env e dashboard/.env.local. A anon key é, por design do Supabase, destinada a ser pública quando RLS está habilitada — mas como F1 mostra que RLS NÃO está habilitada neste projeto, esta chave funciona como uma chave mestra de leitura/escrita para todas as tabelas.
Baixa	.github/workflows/backend-tests.yml:31 .github/workflows/ci.yml:31	F4 — SECRET_KEY de teste fixo nos workflows de CI SECRET_KEY: ci-test-secret está hardcoded nos dois workflows, usado apenas para assinar JWTs durante a execução de testes automatizados contra um Postgres efêmero do próprio job.
Informativa	src/backend/app/scripts/seed.py:73 .github/workflows/deploy.yml (uso de sshpass)	F5 — Senha padrão de seed local ("changeme123") e deploy via SSH com senha seed.py define DEFAULT_PASSWORD = "changeme123" para o admin de dados fictícios de desenvolvimento local (idempotente, cria a empresa 'Salão Anabela' apenas se não existir). Separadamente, o workflow de Deploy autentica no servidor via SSH usando senha (sshpass -e ssh ...) com a senha vinda de um GitHub Secret (SSH_PASSWORD), em vez de chave pública.
Baixa	src/backend/app/routers/notifications.py:46-48	F6 — Token de acesso JWT trafega como query string no endpoint SSE GET /api/notifications/stream?token= recebe o access token via query string em vez do header Authorization, pois EventSource não permite headers customizados.

Achados — detalhe completo por categoria

Banco sem tranca (RLS ausente)

Crítica

F1 — Painel legado (dashboard/) lê dados de todos os tenants sem autenticação e sem RLS

Arquivo(s): dashboard/src/App.jsx:192-194 · dashboard/src/supabaseClient.js:1-6

O painel administrativo legado (Supabase) não possui nenhuma tela de login nem chamada a `supabase.auth` — carrega os dados assim que a página abre. As três queries usam a anon key pública diretamente do navegador e filtram por tenant apenas no JavaScript do cliente (`.eq('project_id', PROJECT_ID)`), nunca no servidor:

```
supabase.from('events').select('*').eq('project_id', PROJECT_ID)...
supabase.from('clients').select('*').eq('project_id', PROJECT_ID)
supabase.from('service_costs').select('*') // sem filtro de tenant
```

Por que é explorável: O próprio CLAUDE.md do projeto documenta que esta SPA legada não tem RLS habilitada. Sem RLS, o PostgREST do Supabase aceita QUALQUER query da anon key, e o filtro `.eq()` é apenas uma conveniência da UI — não uma barreira. Qualquer pessoa com a anon key (pública por natureza, embutida no bundle JS e, além disso, já vazada no histórico do Git — ver F3) pode consultar a REST API do Supabase diretamente (curl/Postman) e ler ou alterar dados de QUALQUER tenant, sem login. A query de `'service_costs'` nem sequer filtra por tenant no próprio código-fonte, expondo dados de custo/margem de todos os tenants a qualquer visitante da página.

Impacto: Vazamento total de nomes e telefones de clientes (PII) e de dados financeiros (custo/preço de serviços) de todos os tenants, sem autenticação.

Correção sugerida: 1) Habilitar Row Level Security em todas as tabelas do Supabase usadas por este app (events, clients, service_costs, booking_requests) com políticas por project_id/tenant. 2) Adicionar autenticação real ao painel (Supabase Auth) antes de qualquer leitura. 3) Girar (rotate) a anon key comprometida e tratar como definitivo — a chave antiga já está no histórico do Git. 4) Dado que o novo stack (src/backend + src/frontend, FastAPI/Postgres) já substituiu este painel, priorizar a descontinuação/remoção de dashboard/ em vez de corrigi-lo.

Chaves expostas (hardcode / histórico Git)

Crítica

F2 — Credenciais reais do Google OAuth (client_secret + refresh_token) commitadas e ainda recuperáveis no histórico do Git

Arquivo(s): credentials_anabela.json (commit 164efb6) · token_anabela.json (commit 164efb6)

O commit 164efb6 ("first commit") adicionou os arquivos `credentials_anabela.json` e `token_anabela.json` com um `client_secret` OAuth real do Google Cloud (projeto 'cabeleireiro-calendario') e um `refresh_token` de escopo `calendar.readonly`. Os arquivos foram removidos de commits posteriores e hoje estão no `.gitignore` (`credential_token/, credentials_*.json`), mas o histórico do Git NÃO foi reescrito — `'git show 164efb6:credentials_anabela.json'` ainda recupera o segredo integralmente para qualquer pessoa com acesso de leitura ao repositório.

Por que é explorável: Remover um arquivo em um commit posterior não apaga seu conteúdo do histórico — ele permanece acessível via `git show/git log -p/git clone` para qualquer colaborador, fork ou vazamento do `.git`. Isso já era conhecido pela equipe: `i-need-to-create-adaptive-donut.md:248` registra explicitamente a tarefa pendente "Remove credentials_anabela.json/token_anabela.json from git history, rotate the underlying Google OAuth credentials... treat the committed ones as compromised regardless of history rewrite" — ou seja, a própria equipe já sinalizou o risco, mas a mitigação (rotação da credencial) não está confirmada como concluída no código.

Impacto: Posse do `client_secret` + `refresh_token` permite a qualquer pessoa gerar access tokens válidos e ler o Google Calendar da conta do proprietário do negócio indefinidamente (refresh tokens não expiram por tempo, só por revogação).

Correção sugerida: 1) Girar IMEDIATAMENTE as credenciais no Google Cloud Console (revogar o client OAuth 873257418071-...apps.googleusercontent.com e o refresh_token associado) — tratar como comprometidas independente de

qualquer reescrita de histórico. 2) Reescrever o histórico do Git (git filter-repo / BFG) para remover os blobs, cientes de que isso não anula a necessidade do passo 1. 3) Nunca versionar arquivos de credenciais — usar variáveis de ambiente (como já é feito em booking-site/api/_lib/googleAuth.js) e confirmar que credentials_*.json/token_*.json seguem no .gitignore em todo o histórico futuro.

Média

F3 — Chave pública (anon/publishable) e URL do Supabase commitadas no histórico do Git

Arquivo(s): .env (commit 224ef52) · dashboard/.env.local (commit f4a3370)

VITE_SUPABASE_URL e VITE_SUPABASE_ANON_KEY (sb_publishable...) foram commitados em .env e dashboard/.env.local. A anon key é, por design do Supabase, destinada a ser pública quando RLS está habilitada — mas como F1 mostra que RLS NÃO está habilitada neste projeto, esta chave funciona como uma chave mestra de leitura/escrita para todas as tabelas.

Por que é explorável: Isoladamente (com RLS correta) isto seria apenas informativo. Combinado com F1 (RLS ausente), vira um vetor de acesso direto às tabelas — e o fato de já estar em texto plano no histórico do Git elimina qualquer dependência de extrair a chave do bundle JS: basta 'git log -p'.

Impacto: Combinado com F1, permite leitura/escrita direta em todas as tabelas do Supabase deste projeto sem passar pela aplicação.

Correção sugerida: Girar a anon key no painel do Supabase assim que a RLS de F1 for implementada, e nunca commitar arquivos .env (o .gitignore atual já bloqueia isso para commits futuros).

Chaves expostas (hardcode)

Baixa

F4 — SECRET_KEY de teste fixo nos workflows de CI

Arquivo(s): .github/workflows/backend-tests.yml:31 · .github/workflows/ci.yml:31

SECRET_KEY: ci-test-secret está hardcoded nos dois workflows, usado apenas para assinar JWTs durante a execução de testes automatizados contra um Postgres efêmero do próprio job.

Por que é explorável: Sem impacto real: o valor nunca assina tokens de produção, e o Postgres do CI é descartado ao final do job. Reportado por completude da varredura de padrões 'secret = '.

Impacto: Nenhum em produção — escopo limitado ao ambiente efêmero do GitHub Actions.

Correção sugerida: Opcional: mover para um GitHub Secret dedicado ao CI por consistência, mas não é uma correção urgente.

Informativa

F5 — Senha padrão de seed local ("changeme123") e deploy via SSH com senha

Arquivo(s): src/backend/app/scripts/seed.py:73 · .github/workflows/deploy.yml (uso de sshpass)

seed.py define DEFAULT_PASSWORD = "changeme123" para o admin de dados fictícios de desenvolvimento local (idempotente, cria a empresa 'Salão Anabela' apenas se não existir). Separadamente, o workflow de Deploy autentica no servidor via SSH usando senha (sshpass -e ssh ...) com a senha vinda de um GitHub Secret (SSH_PASSWORD), em vez de chave pública.

Por que é explorável: O script de seed é claramente rotulado para desenvolvimento local e não é acionável via HTTP. O uso de sshpass com senha (mesmo vinda de secret) é uma prática menos robusta que autenticação por chave SSH — a senha transita como argumento de processo na máquina do runner.

Impacto: Baixo/nenhum para o seed (não é código de produção alcançável). O deploy via senha é uma questão de robustez, não uma chave exposta em código-fonte.

Correção sugerida: Migrar o Deploy workflow para autenticação por chave SSH (deploy key dedicada) em vez de sshpass + senha.

Inputs sem tratamento (dados sensíveis em trânsito)

Baixa

F6 — Token de acesso JWT trafega como query string no endpoint SSE

Arquivo(s): src/backend/app/routers/notifications.py:46-48

GET /api/notifications/stream?token= recebe o access token via query string em vez do header Authorization, pois EventSource não permite headers customizados.

Por que é explorável: Query strings ficam frequentemente registradas em logs de acesso (nginx access_log, já habilitado em infra/nginx/meeting-scheduler.conf), em headers Referer de eventuais navegações subsequentes, e no histórico do navegador — ampliando a superfície de exposição de um token de curta duração (10 min, ver core/config.py).

Impacto: Um access token de 10 minutos vazado por log/histórico permite personificar o usuário até expirar.

Correção sugerida: Documentado como trade-off deliberado (limitação do EventSource) — mitigação razoável: garantir que o access_token_expire_minutes permaneça curto (já é 10 min) e considerar excluir explicitamente esta rota do log de acesso do nginx.

Recomendações priorizadas

Prio.	Ação	Achados
P1	Girar as credenciais Google OAuth comprometidas (client_secret + refresh_token de credentials_anabela.json/token_anabela.json) no Google Cloud Console — ação imediata, independente de qualquer outra correção.	F2
P1	Habilitar Row Level Security em todas as tabelas Supabase usadas por dashboard/ e booking-site (events, clients, service_costs, booking_requests), com políticas por project_id.	F1
P1	Adicionar autenticação real (Supabase Auth ou equivalente) ao painel dashboard/ antes de qualquer leitura de dados.	F1
P2	Girar a anon key do Supabase após a RLS estar em vigor, e reescrever o histórico do Git para remover os blobs de credenciais (cientes de que isso não substitui a rotação).	F2, F3
P2	Priorizar a descontinuação de dashboard/ e booking-site/ em favor do novo stack FastAPI/React, que já resolve o isolamento de tenant corretamente.	F1
P3	Migrar o workflow de Deploy de autenticação SSH por senha (sshpass) para chave SSH dedicada.	F5
P3	Excluir a rota /api/notifications/stream do access_log do nginx, já que o token de acesso trafega na query string.	F6
P3	Mover SECRET_KEY do CI para um GitHub Secret dedicado por consistência (sem risco real atual).	F4

Issues para o GitHub

Texto completo de cada issue, pronto para copiar e colar. Achados triviais relacionados (F4 e F5, ambos de baixo impacto e mesma categoria) foram agrupados em uma única issue.

--- ISSUE 1 ---

[Segurança] Painel legado (dashboard/) expõe dados de todos os tenants sem autenticação (RLS ausente)

Labels sugeridas: security, critical

Problema

O painel administrativo legado em `dashboard/src/App.jsx` não possui tela de login nem chamada a `supabase.auth` – carrega os dados diretamente ao montar o componente, usando a anon key pública do Supabase no navegador. O isolamento por tenant é feito apenas com `.eq('project\_id', PROJECT\_ID)` no JavaScript do cliente, e a query de `service\_costs` nem sequer aplica esse filtro. Como o projeto não tem Row Level Security habilitada (confirmado em CLAUDE.md), o filtro do cliente não é uma barreira real:

qualquer pessoa com a anon key pode consultar a REST API do Supabase diretamente e ler dados de QUALQUER tenant.

Por que é explorável

Sem RLS, o PostgreSQL aceita qualquer query autenticada com a anon key, independente do que a UI decide mostrar. A anon key é pública por natureza (embutida no bundle JS) e, adicionalmente, já está no histórico do Git (ver issue de credenciais).

Evidência

dashboard/src/App.jsx:192-194

...

```
supabase.from('events').select('*').eq('project_id', PROJECT_ID).order('event_start_time',
{ascending:false}),
supabase.from('clients').select('*').eq('project_id', PROJECT_ID),
supabase.from('service_costs').select('*'),
...
```

dashboard/src/supabaseClient.js:1-6 (cliente instanciado só com a anon key, sem sessão)

Impacto

Vazamento de PII de clientes (nome, telefone) e de dados financeiros (custo/preço) de todos os tenants, sem exigir login.

Sugestão de correção

- Habilitar RLS em `events`, `clients`, `service\_costs` e `booking\_requests` com políticas por `project\_id`.
- Adicionar autenticação real ao painel antes de qualquer leitura.
- Girar a anon key do Supabase.
- Avaliar descontinuar `dashboard/` em favor do stack novo (FastAPI + React), que já resolve isolamento de tenant corretamente.

Critérios de aceite

- [] RLS habilitada e testada nas 4 tabelas listadas
- [] Painel exige login válido antes de renderizar qualquer dado
- [] Anon key rotacionada
- [] Consulta direta à REST API do Supabase sem sessão de tenant correto retorna vazio/erro

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Credenciais reais do Google OAuth commitadas e recuperáveis no histórico do Git

Labels sugeridas: security, critical

Problema

O commit `164efb6` ("first commit") incluiu `credentials\_anabela.json` e `token\_anabela.json` com um client_secret OAuth real do Google Cloud e um refresh_token de escopo `calendar.readonly`. Os arquivos foram removidos e hoje estão no `.gitignore`, mas o histórico do Git não foi reescrito – o segredo continua totalmente recuperável.

Por que é explorável

Remover um arquivo em um commit posterior não apaga seu conteúdo do histórico do Git; qualquer pessoa com `git clone`/acesso de leitura ao repositório pode rodar `git show 164efb6:credentials\_anabela.json` e obter o client_secret e o refresh_token na íntegra. Isso já havia sido identificado internamente (`i-need-to-create-adaptive-donut.md:248`), mas não há confirmação de que a rotação foi concluída.

Evidência

- Commit `164efb6`, arquivo `credentials\_anabela.json` (client_secret do projeto Google Cloud "cabeleireiro-calendario")
- Commit `164efb6`, arquivo `token\_anabela.json` (refresh_token com escopo `https://www.googleapis.com/auth/calendar.readonly`)
- Valores reais mascarados neste relatório por prudência – ver os commits para os valores completos.

Impacto

Posse do client_secret + refresh_token permite gerar access tokens válidos e ler o Google Calendar da conta associada indefinidamente, até revogação manual.

Sugestão de correção

- Girar/revogar imediatamente as credenciais no Google Cloud Console – tratar como comprometidas independentemente de qualquer reescrita de histórico.
- Reescrever o histórico do Git (git filter-repo/BFG) para remover os blobs.
- Confirmar que nenhum outro script local usa esses arquivos versionados; usar variáveis de ambiente (padrão já seguido em `booking-site/api/\_lib/googleAuth.js`).

Critérios de aceite

- [] Client OAuth antigo revogado no Google Cloud Console
- [] Novo client/segredo gerado e distribuído apenas via variável de ambiente/secret manager
- [] Histórico do Git reescrito e force-pushed (com aviso à equipe sobre re-clone)
- [] Verificação de que nenhum serviço em produção ainda referencia as credenciais antigas

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Chave pública do Supabase commitada no histórico do Git (agravada pela ausência de RLS)

Labels sugeridas: security, medium

Problema

`VITE\_SUPABASE\_URL` e `VITE\_SUPABASE\_ANON\_KEY` foram commitados em `.env` (commit `224ef52`) e `dashboard/.env.local` (commit `f4a3370`). Isoladamente a anon key é pública por design do Supabase, mas combinada com a ausência de RLS (ver issue do dashboard) ela funciona como uma chave mestra de leitura/escrita.

Por que é explorável

Estar em texto plano no histórico do Git elimina a necessidade de extrair a chave do bundle JS – basta `git log -p` para obtê-la, além de já estar embutida publicamente no frontend.

Evidência

- `.env`, commit `224ef52`: `VITE\_SUPABASE\_URL`, `VITE\_SUPABASE\_ANON\_KEY`
- `dashboard/.env.local`, commit `f4a3370`: mesmos valores

Impacto

Combinado com a issue de RLS ausente, permite leitura/escrita direta em todas as tabelas do projeto Supabase sem passar pela aplicação.

Sugestão de correção

- Girar a anon key assim que a RLS estiver em vigor.
- Confirmar que `.env`/`.env.local` seguem no `.gitignore` (já estão) para todo commit futuro.

```
## Critérios de aceite
- [ ] Anon key rotacionada após RLS habilitada
- [ ] Nenhum novo commit futuro reintroduz arquivos `.env*`
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Segredos de baixo risco: SECRET_KEY fixo no CI e deploy via SSH com senha

Labels sugeridas: security, low

Problema

Dois itens de menor severidade agrupados por serem do mesmo tema (segredos/credenciais em automação, sem exposição de dados de produção):

1. `SECRET\_KEY: ci-test-secret` está hardcoded em `.github/workflows/backend-tests.yml:31` e `.github/workflows/ci.yml:31`, usado apenas para assinar JWTs durante testes contra um Postgres efêmero do próprio job.
2. `.github/workflows/deploy.yml` autentica no servidor de deploy via `sshpass` + senha (vinda de um GitHub Secret `SSH\_PASSWORD`), em vez de autenticação por chave SSH.

Por que é explorável

- (1) Sem impacto real – nunca assina tokens de produção e o ambiente é descartado ao fim do job.
- (2) Autenticação por senha via `sshpass` é uma prática menos robusta que chave SSH – a senha transita como argumento de processo no runner, ainda que vinda de um secret do GitHub.

Evidência

```
- `.github/workflows/backend-tests.yml:31`, `.github/workflows/ci.yml:31`: `SECRET_KEY: ci-test-secret`
- `.github/workflows/deploy.yml`: `sshpass -e ssh -p 1111 ... "${{ secrets.SSH_USER }}"@${{ secrets.HOST }}
```

Impacto

Baixo/nenhum em produção. Item (2) é uma questão de robustez de infraestrutura, não uma chave exposta em código-fonte.

Sugestão de correção

- (1) Opcional: mover para um GitHub Secret dedicado ao CI, por consistência.
- (2) Migrar para autenticação por chave SSH dedicada (deploy key) no lugar de `sshpass` + senha.

Critérios de aceite

- [] SECRET_KEY de CI passa a vir de um secret dedicado (ou justificativa documentada para mantê-lo fixo)
- [] Deploy usa chave SSH em vez de senha

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Token de acesso JWT trafega como query string no endpoint SSE de notificações

Labels sugeridas: security, low

Problema

`GET /api/notifications/stream?token=<jwt>` recebe o access token via query string em vez do header `Authorization`, porque a API `EventSource` do navegador não permite headers customizados.

Por que é explorável

Query strings costumam ficar registradas em logs de acesso (o nginx do projeto já tem `access\_log` habilitado), em headers `Referer` de navegações subsequentes e no histórico do navegador – ampliando a superfície de exposição de um token válido.

Evidência

```
src/backend/app/routers/notifications.py:46-48
```

```
...
```

```
@router.get("/stream")
```

```
async def stream_notifications(request: Request, token: str, db: AsyncSession = Depends(get_db)):
```

```
    user = await _get_sse_user(token, db)
```

```
...
```

Impacto

Um access token vazado por log/histórico permite personificar o usuário até expirar. O risco é parcialmente mitigado pela curta duração do token (`access\_token\_expire\_minutes = 10`, ver `core/config.py`).

Sugestão de correção

- Excluir explicitamente esta rota do `access\_log` do nginx (`infra/nginx/meeting-scheduler.conf`, location `/api/notifications/stream`).
- Manter o `access\_token\_expire\_minutes` curto (já está em 10 minutos).

Critérios de aceite

- [] Location do nginx para `/api/notifications/stream` desabilita access_log
- [] Confirmado que o token continua expirando em <= 10 minutos

--- FIM ISSUE 5 ---